

JAVA OOP TECHNICAL INTERVIEW HANDBOOK

Complete, High-Yield Preparation & Quick-Revision Guide for SAP Labs & Tech Interviews

Target Role: Software Development Engineer (Fresher) **Focus:** Core OOP, Java Memory, Traps & SOLID **Format:** High-Density Revision Handbook

TABLE OF CONTENTS

1. OOP Fundamentals	16. Object Class & Methods
2. Four Pillars of OOP	17. equals() and hashCode() Contract
3. Classes, Objects & Equality	18. Association, Aggregation, Composition
4. Constructors & Chaining	19. Designing Immutable Classes
5. The 'this' Keyword	20. Java Memory Model for OOP
6. Inheritance Mechanics	21. Upcasting, Downcasting & ClassCastException
7. The 'super' Keyword	22. Coupling, Cohesion & Dependency Injection
8. Polymorphism & Dynamic Dispatch	23. Composition vs Inheritance
9. Method Overloading Rules	24. SOLID Principles (Java Examples)
10. Method Overriding Rules	25. Top 30 Java Interview Traps
11. Abstraction & Abstract Classes	26. 25 Tricky Output-Based Code Snippets
12. Interfaces (Default, Static, Private)	27. Categorized Interview Questions & Answers
13. Access Modifiers Matrix	28. Rapid Revision Cheat Sheet
14. The 'static' Keyword & Init Order	29. 30-Minute Last-Minute Revision
15. The 'final' Keyword & Immutability	30. Professional Answering Strategy

PART 1 — OOP FUNDAMENTALS

1.1 Basic Core Concepts

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which contain data (fields/attributes) and code (methods/behaviors). Instead of focusing on functions operating on data, OOP structures software around entities that represent real-world or conceptual objects.

Why OOP?

- **Modularity:** Code is partitioned into distinct objects, making it easier to maintain and debug.
- **Reusability:** Through inheritance and composition, code can be reused across projects.
- **Maintainability:** Changes inside an object (encapsulation) do not affect other parts of the system.
- **Extensibility:** New functionality can be added with minimal changes to existing code (polymorphism).

1.2 Procedural vs Object-Oriented Programming

Feature	Procedural Programming (C, Pascal)	Object-Oriented Programming (Java, C++)
Primary Focus	Functions/Functions operating on data.	Data and objects bundling data with functions.
Structure	Top-down approach.	Bottom-up approach.
Data Security	Low (Data moves freely around system).	High (Encapsulation protects data access via modifiers).
Reusability	Low (Function copy-pasting, header files).	High (Inheritance, interfaces, polymorphic reuse).

1.3 Class, Object, State, Behavior, Identity

- **Class:** A blueprint, template, or user-defined data type from which objects are created. It consumes memory in the Metaspace/Method Area when loaded by the ClassLoader, but does NOT occupy Heap memory for instances until instantiated.
- **Object:** A runtime instance of a class occupying memory in the Heap space.
- **State:** Represented by the instance variables (data) of an object at a given point in time.
- **Behavior:** Represented by the methods of an object, defining actions it can perform.
- **Identity:** A unique identity (memory address / internal hash) assigned by JVM to distinguish it from other objects, even if their state is identical.

```
+-----+
| CLASS: Car (Blueprint in Method Area) |
| Fields: String model, int speed | Methods: accelerate(), brake() |
+-----+
                |
        new Car("BMW", 120); | new Car("BMW", 120);
                v
+-----+ +-----+
| OBJECT 1 (Heap - Addr: 0x101) | | OBJECT 2 (Heap - Addr: 0x102) |
| State: model="BMW", speed=120 | | State: model="BMW", speed=120 |
| Identity: 0x101                | | Identity: 0x102                |
+-----+ +-----+
```

```

public class Car {
    // Instance variables (State)
    private String model;
    private int speed;

    // Static variable (Class-level state)
    public static int totalCarsProduced = 0;

    // Constructor
    public Car(String model, int speed) {
        this.model = model;
        this.speed = speed;
        totalCarsProduced++;
    }

    // Instance Method (Behavior)
    public void accelerate(int increment) {
        this.speed += increment;
    }
}

```

⚠ INTERVIEW TRAP — STATIC VS INSTANCE MEMBERS

Instance variables exist **per object** on the Heap. Static variables exist **once per class** in the Method Area / Metaspace. Calling a static method on a null reference variable (e.g., `Car c = null; c.getCarCount();`) does NOT throw a `NullPointerException` because static resolution is performed at compile time using the reference's declared type.

1.4 Object Lifecycle Basics

- Class Loading:** The `ClassLoader` loads the `.class` byte code into memory. Static blocks execute and static variables are initialized.
- Instantiation:** Memory is allocated on the Heap using `new` keyword. Default values assigned (0, null, false).
- Initialization:** Instance initializers and constructors execute.
- In-Use:** Object reference is actively held on stack/heap and manipulated.
- Unreachable / Garbage Collection:** Object loses all active references. The Garbage Collector (GC) marks it for deletion and reclaims memory.

PART 2 — FOUR PILLARS OF OOP

2.1 Pillars Summary Matrix

Concept	Meaning	Java Mechanism	Main Purpose	Real-World Example
Encapsulation	Wrapping data & code into single unit; hiding internal state.	<code>private</code> fields + <code>public</code> getters/setters.	Data protection, control, validation.	Medical Capsule / ATM Machine interface.
Abstraction	Hiding implementation complexity; showing essential features.	<code>abstract</code> classes, <code>interface</code> .	Reduce complexity, set clear contracts.	Car accelerator pedal (don't care how engine ignites).
Inheritance	Mechanism where child class acquires properties of parent class.	<code>extends</code> keyword.	Code reusability, IS-A hierarchy.	Dog IS-A Mammal IS-A Animal.
Polymorphism	Ability of an entity to take multiple forms.	Method Overloading & Overriding (Dynamic Dispatch).	Flexibility, loose coupling.	Power button (boots TV, Laptop, or Phone differently).

2.2 Deep Dive into Each Pillar

A. Encapsulation

- **Why it exists:** To prevent direct, unauthorized modification of instance fields. Protects invariant states.
- **Java Implementation:** Declare fields `private` and expose controlled access via public methods.
- **Misconception:** "Encapsulation is just writing getters and setters for every variable." False — if a setter allows setting negative age without validation, encapsulation is broken. Encapsulation is about maintaining object integrity.

```
public class BankAccount {  
    private double balance; // Hidden internal state  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            this.balance += amount; // Validation enforced  
        }  
    }  
}
```

B. Abstraction

- **Abstraction vs Encapsulation:** Abstraction hides *complexity* (at design level). Encapsulation hides *data* (at implementation level).
- **When to use:** When defining a standard template/contract for a family of implementations (e.g., `PaymentProcessor`).

C. Inheritance

- **Advantages:** Code reuse, subtype polymorphism, clear conceptual hierarchies.
- **Disadvantages:** Tight coupling (fragile base class problem). Modifying parent breaks children.
- **Java Constraint:** Java does NOT support multiple inheritance with classes to prevent the **Diamond Problem**.

DIAMOND PROBLEM (Why Java restricts multiple inheritance with classes):

```
    [Class A]  
    /      \ [Class B] [Class C] (Both override method foo())  
   \      /  
  [Class D]      (Ambiguity: Which foo() does D inherit? Compile Error!)
```

D. Polymorphism

- **Compile-Time (Static):** Resolved by compiler using method signature (Method Overloading).
- **Runtime (Dynamic):** Resolved at runtime using actual object type in Heap via Dynamic Method Dispatch (Method Overriding).

PART 3 — CLASSES AND OBJECTS

3.1 Reference Equality vs Object Equality

- `==` operator compares reference variables for **memory addresses** (whether they point to the exact same object on Heap).
- `equals()` method compares **logical equality** (content/state), provided it is overridden in the class.

```
String s1 = new String("SAP");
String s2 = new String("SAP");

System.out.println(s1 == s2);      // false (Different heap objects)
System.out.println(s1.equals(s2)); // true  (Same string content)

Object a = new Object();
Object b = a; // Copy reference
System.out.println(a == b);      // true (Point to exact same Heap address)
```

3.2 Mutable vs Immutable Objects

- **Mutable:** State can be modified after creation (e.g., `StringBuilder`, `ArrayList`, `Date`).
- **Immutable:** State cannot be changed once created (e.g., `String`, all Primitive Wrapper classes like `Integer`, `Double`).

★ ANONYMOUS OBJECTS

An object created without storing its reference in a variable: `new Student().study();` . Used for one-time method calls. Immediately becomes eligible for Garbage Collection after execution completes.

PART 4 — CONSTRUCTORS

4.1 Constructor Essentials

A constructor is a special block of code called when an instance of an object is created using `new` . It initializes the object's state.

- Must have the **exact same name** as the class.
- Must **NOT have any return type** (not even `void`). If you add `void` , Java treats it as a standard method, NOT a constructor!
- If NO constructor is declared in a class, the compiler automatically inserts a **default no-arg constructor** with an empty body and access modifier matching the class.
- If ANY constructor is defined (e.g., parameterized), the compiler does **NOT** generate the default no-arg constructor.

⚠ INTERVIEW TRAP — CONSTRUCTOR COMPILATION ERROR

```
class Person {
    Person(String name) {} // Parameterized constructor present
}
public class Test {
    public static void main(String[] args) {
        Person p = new Person(); // COMPILER ERROR! No default constructor exists.
    }
}
```

4.2 Constructor Traps Q&A Matrix

Question	Answer	Explanation / Reason
Is constructor inherited?	NO	Constructors belong to the class in which they are declared. Subclasses invoke them via <code>super()</code> .
Can constructor be overridden?	NO	Overriding requires inheritance. Since constructors are not inherited, they cannot be overridden.
Can constructor be static?	NO	Static members belong to the class. Constructors are executed to create an instance.
Can constructor be final?	NO	Final prevents overriding in subclasses. Since constructors cannot be inherited, final is illegal.
Can constructor be abstract?	NO	Abstract means implementation is deferred. A constructor must have an execution body to initialize fields.
Can constructor be private?	YES	Used in Singleton design pattern or utility classes to prevent external instantiation.

4.3 Execution Order on Instantiation

1. Static Variables & Static Initialization Blocks of Parent Class (in textual order).
2. Static Variables & Static Initialization Blocks of Child Class (in textual order).
3. Instance Variables & Instance Initializer Blocks of Parent Class.
4. Parent Class Constructor execution.
5. Instance Variables & Instance Initializer Blocks of Child Class.
6. Child Class Constructor execution.

```
class Parent {
    static { System.out.println("1. Parent Static Block"); }
    { System.out.println("3. Parent Instance Block"); }
    Parent() { System.out.println("4. Parent Constructor"); }
}

class Child extends Parent {
    static { System.out.println("2. Child Static Block"); }
    { System.out.println("5. Child Instance Block"); }
    Child() { System.out.println("6. Child Constructor"); }

    public static void main(String[] args) {
        new Child();
    }
}
```

PART 5 — THIS KEYWORD

5.1 Six Primary Uses of `this`

1. Refer to current class instance variable: `this.variable` (resolves variable shadowing).
2. Invoke current class method: `this.method()`.
3. Invoke current class constructor: `this()` or `this(args)` (Constructor chaining).
4. Pass as an argument in a method call: `logger.log(this)`.
5. Pass as an argument in a constructor call: `new EventListener(this)`.
6. Return current class instance from a method: `return this;` (used in Builder Pattern).

⚠ FIRST STATEMENT RULE

Calls to `this()` or `super()` within a constructor **MUST be the very first statement**. Consequently, `this()` and `super()` cannot both be present in the same constructor body.

PART 6 — INHERITANCE

6.1 IS-A vs HAS-A Relationships

- **IS-A Relationship (Inheritance):** Implemented using `extends` or `implements`.

Example: `Dog` IS-A `Animal`.

- **HAS-A Relationship (Composition / Aggregation):** One class holds a reference to another class as a member variable.

Example: `Car` HAS-A `Engine`.

6.2 Types of Inheritance Supported in Java

Type	Supported for Classes?	Supported for Interfaces?
Single Inheritance	Yes (<code>class B extends A</code>)	Yes (<code>interface B extends A</code>)
Multilevel Inheritance	Yes (<code>class C extends B extends A</code>)	Yes
Hierarchical Inheritance	Yes (<code>class B extends A</code> , <code>class C extends A</code>)	Yes
Multiple Inheritance	NO (Prevents ambiguity/Diamond problem)	YES (via multiple interfaces)
Hybrid Inheritance	NO (Using classes alone)	YES (Using interfaces)

PART 7 — SUPER KEYWORD

7.1 Uses of `super`

- `super.field` : Accesses parent class variable hidden/shadowed by child class field.
- `super.method()` : Calls overridden parent class method.
- `super()` : Invokes direct parent class constructor.

```
class Parent {
    String name = "ParentName";
    void show() { System.out.println("Parent Method"); }
}

class Child extends Parent {
    String name = "ChildName";

    @Override
    void show() {
        super.show(); // Calls Parent's show()
        System.out.println("Child Name: " + this.name);
        System.out.println("Parent Name: " + super.name);
    }
}
```

PART 8 — POLYMORPHISM & DYNAMIC METHOD DISPATCH

8.1 Crucial Polymorphic Reference Concept

Consider the classic interview construct:

```
Parent p = new Child();
```

When this line executes, understand the separation between **Reference Type** and **Object Type**:

Reference Variable: p (Type: Parent) -----> Object in Heap: [Type: Child]
(Determines Accessible Members at Compile Time) (Determines Overridden Method Execution at Runtime)

Member Type	Resolution Rule	What Gets Accessed / Executed?
Instance Methods (Overridden)	Resolved at Runtime based on Object Type in Heap.	Executes Child's overridden method! (Dynamic Method Dispatch)
Instance Variables / Fields	Resolved at Compile-Time based on Reference Type .	Accesses Parent's field! (Variables are NOT polymorphic)
Static Methods	Resolved at Compile-Time based on Reference Type .	Executes Parent's static method! (Method Hiding)

```
class Parent {
    int x = 10;
    void display() { System.out.println("Parent Method"); }
    static void print() { System.out.println("Parent Static"); }
}

class Child extends Parent {
    int x = 20;
    @Override
    void display() { System.out.println("Child Method"); }
    static void print() { System.out.println("Child Static"); }
}

public class Test {
    public static void main(String[] args) {
        Parent p = new Child();
        System.out.println(p.x); // OUTPUT: 10 (Field from Parent reference)
        p.display();             // OUTPUT: Child Method (Dynamic Dispatch)
        p.print();               // OUTPUT: Parent Static (Method Hiding)
    }
}
```

PART 9 & 10 — METHOD OVERLOADING VS METHOD OVERRIDING

9.1 Core Rules Comparison

Property	Method Overloading	Method Overriding
Definition	Same method name with different parameter list in same class.	Child class provides specific implementation of parent method.
Location	Same class (or inherited).	Across Parent and Child subclasses.
Parameters	Must be different (type, count, or order).	Must be exact same.
Return Type	Can be same or different.	Must be same or Covariant (subclass type).
Access Modifiers	Can be anything.	Cannot be made more restrictive in child class.
Exceptions	Can throw any exception.	Child cannot declare new or broader checked exceptions .
Binding Time	Compile-time (Static binding).	Runtime (Dynamic binding).
Private/Final/Static	Can be overloaded.	Cannot be overridden!

9.2 Overloading Resolution Order (Specificity Hierarchy)

When resolving overloaded methods, Java compiler selects the method using the following strict order of priority:

1. **Exact Primitive Type Match**
2. **Implicit Primitive Widening / Type Promotion** (e.g., `int` → `long` → `float` → `double`)
3. **Autoboxing** (e.g., `int` → `Integer`)
4. **Varargs** (e.g., `int...`)

```
public class OverloadTest {
    static void fun(int x) { System.out.println("int"); }
    static void fun(long x) { System.out.println("long"); }
    static void fun(Integer x) { System.out.println("Integer"); }
    static void fun(int... x) { System.out.println("varargs"); }
    static void fun(Object x) { System.out.println("Object"); }
    static void fun(String x) { System.out.println("String"); }

    public static void main(String[] args) {
        fun(5);           // Output: int (Exact match)
        fun(null);        // Output: String! (String is more specific than Object)
    }
}
```

⚠️ AMBIGUOUS NULL CALL TRAP

If you have `fun(String s)` and `fun(Integer i)`, calling `fun(null)` causes a **Compile-Time Error** because `String` and `Integer` are at the exact same level in the inheritance hierarchy (both direct subclasses of `Object`), so Java cannot decide which is more specific!

PART 11 & 12 — ABSTRACTION: ABSTRACT CLASSES VS INTERFACES

11.1 Comparative Analysis

Feature	Abstract Class	Interface (Java 8+)
Instantiation	Cannot be instantiated directly.	Cannot be instantiated directly.
Multiple Inheritance	No (Class extends single parent).	Yes (Class implements multiple interfaces).
Variables	Can have instance, static, final, non-final fields.	Implicitly <code>public static final</code> (Constants).
Methods	Abstract and concrete methods.	Abstract, <code>default</code> (Java 8), <code>static</code> (Java 8), <code>private</code> (Java 9).
Constructors	YES (Invoked by subclasses via <code>super()</code>).	NO constructors allowed.
Access Modifiers	Methods can be public, protected, package-private.	Interface methods are implicitly <code>public</code> .

12.1 Interface Diamond Conflict Resolution (Java 8 Default Methods)

When a class implements two interfaces that declare identical `default` method signatures:

```
interface A {
    default void show() { System.out.println("A"); }
}
interface B {
    default void show() { System.out.println("B"); }
}

class C implements A, B {
    // MUST explicitly resolve conflict by overriding show()
    @Override
    public void show() {
        A.super.show(); // Calls A's default method implementation explicitly
    }
}
```

PART 13 — ACCESS MODIFIERS MATRIX

Access Modifier	Same Class	Same Package Subclass	Same Package Non-Subclass	Different Package Subclass	Different Package Non-Subclass
<code>private</code>	Yes	No	No	No	No
<code>default</code> (package-private)	Yes	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes	Yes (via inheritance only)	No
<code>public</code>	Yes	Yes	Yes	Yes	Yes

⚠️ PROTECTED MODIFIER NUANCE

In a different package, a subclass can access a `protected` member of the parent class **ONLY through an instance of the subclass** (or its children), NOT through a reference of the Parent class directly!

PART 14 & 15 — STATIC, FINAL, AND IMMUTABILITY

14.1 Static Rules Summary

- Static variables/methods belong to the **Class**, not individual instances.
- Static methods **CANNOT access instance variables or call instance methods directly** (because there is no implicit `this` reference in static context).
- Static methods **CANNOT use `this` or `super` keywords**.

15.1 Final Keyword Breakdown

- `final` variable: Value cannot be reassigned after initialization (constant).
- `final` reference variable: Pointing memory address cannot change, but the internal state of the referenced object **CAN STILL MUTATE!**
- `final` method: Cannot be **overridden** by subclasses.
- `final` class: Cannot be **extended / inherited** (e.g., `java.lang.String` is `final`).

```
final ArrayList<String> list = new ArrayList<>();  
list.add("SAP"); // VALID! Content of object mutates.  
// list = new ArrayList<>(); // COMPILE ERROR! Cannot reassign final reference variable.
```

PART 16 & 17 — OBJECT CLASS, EQUALS & HASHCODE CONTRACT

16.1 The `java.lang.Object` Core Methods

Every class in Java directly or indirectly inherits from `java.lang.Object`. Key methods:

- `public boolean equals(Object obj)` : Checks logical equality. Default implementation uses `==`.
- `public int hashCode()` : Returns hash code integer for hashing datastructures (like `HashMap`, `HashSet`).
- `public String toString()` : Default returns `getClass().getName() + "@" + Integer.toHexString(hashCode())`.
- `protected Object clone()` : Creates copy of object (requires `Cloneable` interface).
- `public final Class<?> getClass()` : Returns runtime class object.

17.1 The `equals()` and `hashCode()` Contract Rules

★ MANDATORY INTERVIEW CONTRACT RULES

1. If two objects are **equal according to `equals()`**, calling `hashCode()` on each **MUST** produce the **same integer result**.
2. If two objects produce the **same `hashCode()`**, they are **NOT required to be equal** (this is known as a **hash collision**).
3. Therefore, if you override `equals()`, you **MUST override `hashCode()`** !

Consequence of Breaking Contract: If `equals()` is overridden without `hashCode()`, custom objects used as keys in a `HashMap` or elements in a `HashSet` will fail to be retrieved, leading to duplicate entries or null returned on lookup!

PART 18 & 19 — ASSOCIATION, AGGREGATION, COMPOSITION & IMMUTABILITY

18.1 Relationship Classification

- **Aggregation (Weak HAS-A):** Child object can exist independently of parent.
Example: `Department` and `Teacher`. If `Department` is deleted, `Teacher` still exists.
- **Composition (Strong HAS-A):** Child object cannot exist without parent.
Example: `Car` and `Engine` / `House` and `Room`. If `House` is destroyed, `Rooms` are destroyed.

19.1 How to Design a Custom Immutable Class in Java

1. Declare class as `final` (prevents subclassing and method overriding).
2. Make all fields `private` and `final` (prevents direct access and re-assignment).
3. Do NOT provide setter methods.
4. Initialize all fields via constructor performing **Defensive Copying** for mutable fields.
5. Return **Defensive Copies** in getter methods for mutable objects.

```
import java.util.Date;

public final class ImmutableStudent {
    private final String name;
    private final Date joiningDate; // Mutable object

    public ImmutableStudent(String name, Date joiningDate) {
        this.name = name;
        // Defensive Copying during construction
        this.joiningDate = new Date(joiningDate.getTime());
    }

    public String getName() { return name; }

    public Date getJoiningDate() {
        // Defensive Copying during access
        return new Date(joiningDate.getTime());
    }
}
```

PART 20 & 21 — JAVA MEMORY MODEL & TYPE CASTING

20.1 Memory Breakdown for Object Reference

```
Person p = new Person();
```

- **Stack:** Stores local variable `p` (which holds reference address/pointer).
- **Heap:** Stores actual `Person` object instance and its instance variables.
- **Metaspace / Method Area:** Stores class structure metadata, method code, static variables, and constant pool.

21.1 Upcasting vs Downcasting

- **Upcasting (Safe):** Casting child object to parent reference. Done automatically.
`Parent p = new Child();`
- **Downcasting (Unsafe):** Casting parent reference back to child reference. Requires explicit cast.
`Child c = (Child) p;`

⚠ CLASSCASTEXCEPTION TRAP

If the underlying object in Heap is NOT an instance of the target downcast type, JVM throws a runtime `ClassCastException`. Always protect downcasts using `instanceof` checks!

```
Parent p = new Parent();
if (p instanceof Child) {
    Child c = (Child) p; // Safe check
}
```

PART 22 & 23 — COUPLING, COHESION & COMPOSITION VS INHERITANCE

22.1 Design Principles

- **Cohesion:** Measure of how focused a single module/class is on a single purpose. Goal: **High Cohesion**.
- **Coupling:** Degree of interdependence between software modules. Goal: **Loose Coupling**.

23.1 Why "Favor Composition Over Inheritance"?

1. Inheritance breaks encapsulation because child classes depend on parent implementation details (Fragile Base Class problem).
2. Inheritance is static (decided at compile-time); Composition is dynamic (can swap components at runtime).
3. Avoids deep hierarchical class pollution.

PART 24 — SOLID PRINCIPLES FOR JAVA INTERVIEWS

Principle	Core Concept	Violating Pattern	Corrected Java Design
S - Single Responsibility	A class should have one, and only one, reason to change .	<code>User</code> class containing DB save logic and email formatting.	Split into <code>User</code> , <code>UserRepository</code> , and <code>EmailService</code> .
O - Open/Closed	Software entities should be open for extension, closed for modification .	Using <code>if-else</code> checks on <code>Shape</code> type to calculate area in single method.	Create <code>Shape</code> interface with <code>calculateArea()</code> method.
L - Liskov Substitution	Derived classes must be substitutable for their base classes without breaking system.	<code>Square</code> extending <code>Rectangle</code> where setting width breaks height.	Separate base abstractions or use composition.
I - Interface Segregation	Clients should not be forced to depend on interfaces they do not use.	Monolithic <code>Worker</code> interface with <code>work()</code> and <code>eat()</code> for <code>Robot</code> .	Split into <code>Workable</code> and <code>Eatable</code> interfaces.
D - Dependency Inversion	Depend upon abstractions, not concretions .	High-level <code>Car</code> directly instantiating specific <code>V8Engine</code> class inside constructor.	Inject <code>Engine</code> interface instance into <code>Car</code> via constructor.

PART 25 — TOP JAVA OOP INTERVIEW TRAPS

TRAP 1: PRIVATE METHODS CANNOT BE OVERRIDDEN

Private methods are bounded at compile time (static binding). Declaring a method with the same signature in a child class creates an entirely NEW independent method, NOT an override!

TRAP 2: STATIC METHOD HIDING

If a child class defines a static method with the exact signature as a static method in parent class, it is called **Method Hiding**, NOT overriding. The method called depends on reference type, not object type.

TRAP 3: OVERRIDING RETURN TYPES & COVARIANCE

An overriding method can return a subtype of the return type declared in parent method (Covariant Return Type), but CANNOT return a broader supertype or unrelated type.

TRAP 4: EXCEPTION RULES IN OVERRIDING

An overriding method can throw fewer or narrower **Checked exceptions** than parent method, or no checked exceptions at all. It CANNOT throw new or broader checked exceptions! (Unchecked exceptions have no restriction).

TRAP 5: FIELD HIDING

Fields in Java CANNOT be overridden. If a child class declares a variable with the same name as parent class variable, the parent variable is hidden. Field access is always resolved based on **Reference Type**.

PART 26 — 25 TRICKY OUTPUT-BASED CODE SNIPPETS

Snippet 1: Dynamic Dispatch vs Field Resolution

```
class A { int x = 10; void m() { System.out.println("A"); } }
class B extends A { int x = 20; void m() { System.out.println("B"); } }

public class Test1 {
    public static void main(String[] args) {
        A obj = new B();
        System.out.print(obj.x + " ");
        obj.m();
    }
}
```

Options: A) 10 A | B) 20 B | C) 10 B | D) Compile Error

Answer: C) 10 B

Explanation: Field `x` is resolved at compile time from reference type `A` (value 10). Method `m()` is resolved dynamically at runtime from object type `B`.

Snippet 2: Overloading with Null Arguments

```
public class Test2 {
    static void test(Object o) { System.out.println("Object"); }
    static void test(String s) { System.out.println("String"); }

    public static void main(String[] args) {
        test(null);
    }
}
```

Options: A) Object | B) String | C) Compile Error | D) NullPointerException

Answer: B) String

Explanation: Java selects the most specific method. Since `String` is a subclass of `Object`, `String` is more specific.

Snippet 3: Static Method Calling on Null Reference

```
class Helper {
    static void print() { System.out.println("Static Print"); }
}
public class Test3 {
    public static void main(String[] args) {
        Helper h = null;
        h.print();
    }
}
```

Options: A) Static Print | B) NullPointerException | C) Compile Error

Answer: A) Static Print

Explanation: Static methods are resolved at compile-time using reference type `Helper`. The actual runtime reference value (`null`) is ignored.

Snippet 4: Constructor Exception Rule

```
class Parent {
    Parent() throws Exception { System.out.println("Parent"); }
}
class Child extends Parent {
    Child() { System.out.println("Child"); }
}
```

Answer: Compile Error!

Explanation: Implicit `super()` call in `Child` constructor must handle/declare checked `Exception` declared in `Parent` constructor!

PART 27 — CATEGORIZED INTERVIEW QUESTIONS & ANSWERS

27.1 SAP Labs Specific & Fresher Questions

Q1: Why is Multiple Inheritance supported via Interfaces but not Classes in Java?

Expected Answer: To avoid the *Diamond Problem* (ambiguity in inheriting multiple state/implementation copies). Classes contain state (instance fields), which creates memory layout ambiguity. Interfaces historically had no state or implementation. With Java 8 default methods, diamond ambiguity is resolved at compile time by forcing implementing classes to override conflicting default methods.

Q2: Can we override a static method? What happens if we write the same static method in Child class?

Expected Answer: No, static methods cannot be overridden because static methods are bound at compile time using Class type (Static Binding), whereas overriding requires runtime dynamic dispatch based on Object type. Writing the same static method signature in a child class is called **Method Hiding**.

Q3: What is the difference between shallow copy and deep copy?

Expected Answer:

- **Shallow Copy:** Copies primitive fields directly, but copies *references* for object fields (both original and copied object share referenced inner objects).

- **Deep Copy:** Recursively creates duplicates of both top-level objects and all nested referenced objects, resulting in total isolation.

PART 28 — RAPID REVISION CHEAT SHEET

⚡ HIGH-YIELD CHEAT SHEET

- `Parent p = new Child();` → Fields from `Parent`, Overridden Methods from `Child`.
- `this()` and `super()` must be 1st line in constructor. Cannot coexist in same constructor.
- `private`, `static`, and `final` methods CANNOT be overridden.
- Abstract classes CAN have constructors; Interfaces CANNOT.
- Interface fields: Implicitly `public static final`. Interface methods: Implicitly `public`.
- Covariant return types allowed during overriding (subtype of parent method return type).
- Overriding method cannot restrict access modifier or declare broader checked exceptions.
- Equal objects MUST have equal HashCodes. Equal HashCodes do NOT guarantee equal objects.

PART 29 — 30-MINUTE LAST-MINUTE REVISION

Top 10 Instant Recalls Before Entering Interview Room

1. **Encapsulation:** Data hiding + restricted access (private fields, public methods).
2. **Abstraction:** Hiding implementation details using Abstract Classes / Interfaces.
3. **Polymorphism:** Compile-time (Overloading) vs Runtime (Overriding / Dynamic Dispatch).
4. **Inheritance:** Reusability using `extends`. Single class inheritance only.
5. **Method Hiding:** Redefining a static method in child class. Reference type dictates call.
6. **Constructor Order:** Static blocks → Parent Instance Blocks & Constructor → Child Instance Blocks & Constructor.
7. **Overloading Null Ambiguity:** Compiler selects most specific class type; equal hierarchy depth throws compile error.
8. **Immutable Class Recipe:** `final` class, `private final` fields, no setters, defensive copies in constructor/getters.
9. **equals() & hashCode():** Always override together to avoid breaking HashMap/HashSet contracts.
10. **SOLID:** Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.

PART 30 — PROFESSIONAL INTERVIEW ANSWERING STRATEGY

The 4-Step Answering Framework

When asked conceptual OOP questions in technical interviews (e.g., at SAP Labs), structured delivery distinguishes strong candidates:

1. **1-Sentence Core Definition:** Direct, precise answer without stuttering.
2. **Java Implementation Mechanism:** Exact syntax / keywords used in Java (e.g., `extends`, `@Override`, `dynamic dispatch`).
3. **Practical 5-Second Real-World Example:** Connect theory to code or architecture.
4. **Edge Case / Interview Trap Mention:** Proactively call out a common trap before the interviewer asks (demonstrates deep mastery).

EXAMPLE ANSWER TEMPLATE: "EXPLAIN RUNTIME POLYMORPHISM"

"Runtime polymorphism in Java is the mechanism where a call to an overridden method is resolved dynamically at runtime based on the actual object type in Heap memory, rather than the reference type. In Java, this is implemented using method overriding where a subclass overrides a parent method marked with the same signature. For example, if a `Shape` reference points to a `Circle` object on Heap, calling `shape.draw()` executes `Circle`'s `draw()` method via Dynamic Method Dispatch. An important interview edge case here is that field access and static methods are NOT polymorphic — they are always bound at compile time based on the reference type."